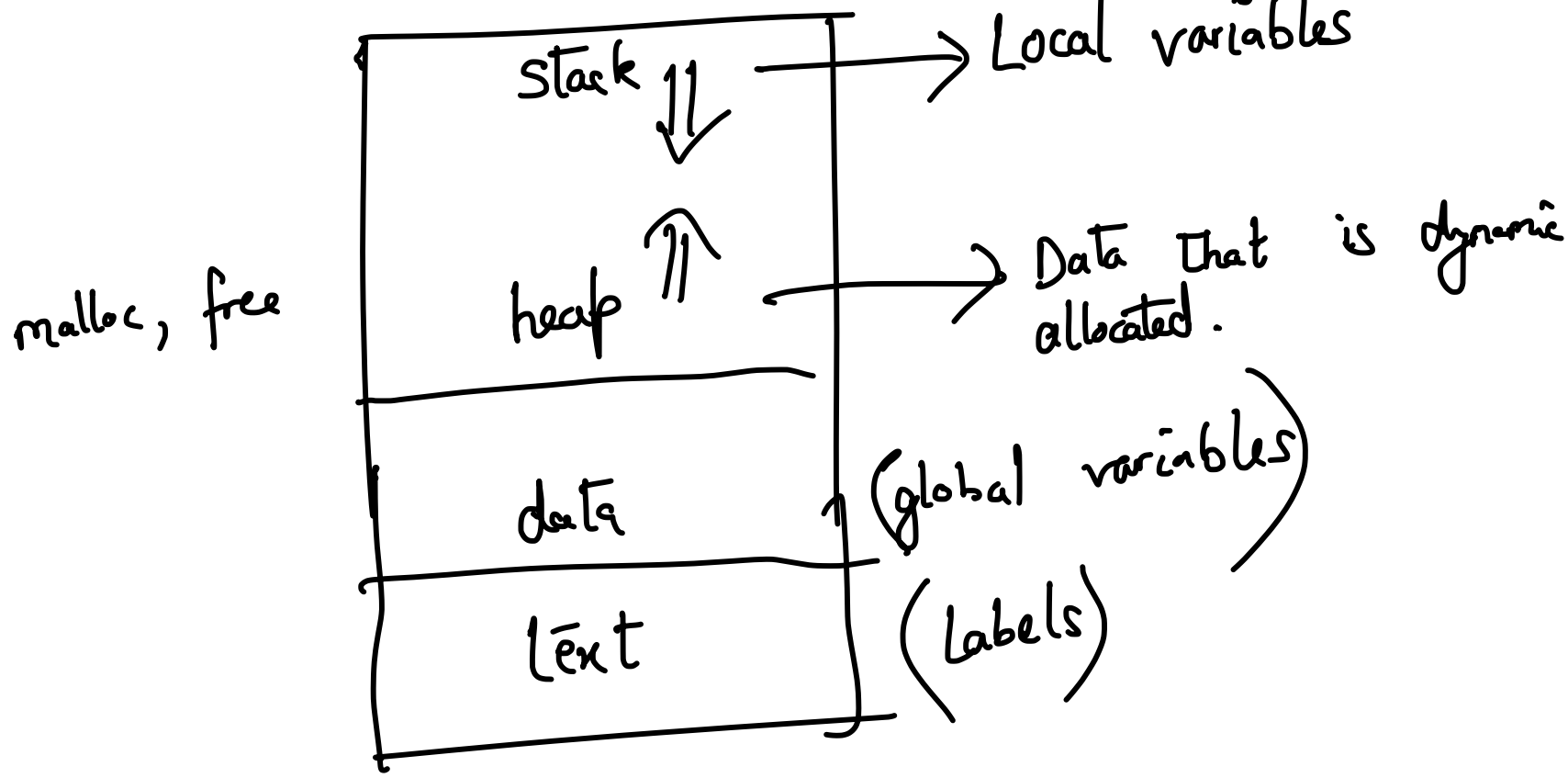
Assembly Programming

Manipulate registers
Perform any arithmetic or logical operations
If conditions / Loops



push and pop instructions

push rax

First subtract 8 from rsp and then place whatever is present in rax in this address

During function calls, stack need to be aligned.
rsp should have a value that is divisible by 16.
You are likely to get a segmentation fault if it is not aligned.

Alignment is necessary to satisfy the HW constraints.

push next_instruction

my_function:

pop rsp [rsp]

next_instruction:

One way of calling a function

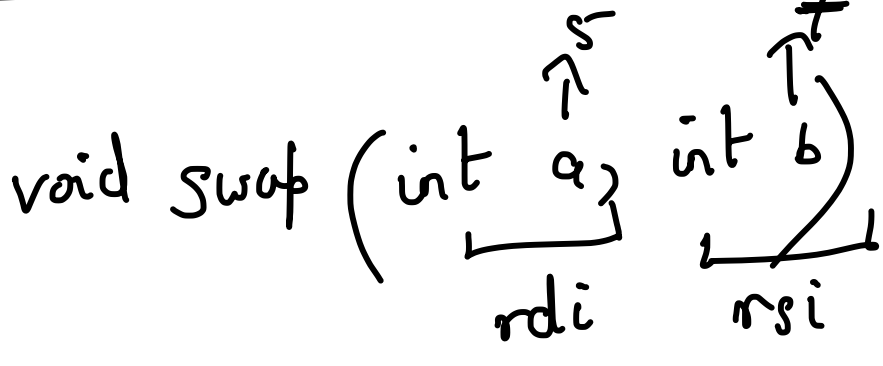
call my_function

ret

Calling convention

1st argument → rdi

2nd " → rsi



3rd → rdx

4th → rcx

5th → r8

6th → r9

7th → stack → rsp - 8

```
mov rdi, 5
mov rsi, 7
call swap
```

rax register for return values

xmm0, ...) xmm7
1st 8th

rax register would also contain the number of floating point arguments.

.text

str: db "Hello world!", 0xa, 0

extern printf

Tell the assembler is not in the file itself, but in either some other

Hello world!\n\0

print.asm

```
lea rdi, [str]
mov rax, 0
call printf
```

file or in a library.

```
yasm -f elf64 print.asm
ld -o print print.asm -.
```

```
str:    "Hello World! %d", 0x0a, 0

        lea rdi, [str]
        mov rsi, 5
        mov rax, 0
        call printf
```

lea ⟶ load effective address

Constants : equ pseudo-op

Times pseudo-op

```
dw a 0
dw a times 30 0
```

⟶ define array using this technique

What is a system call?

⟶ System calls are not treated as ordinary functions

32-bit and 64-bit system.

Store the parameter for the system call in the rax register and then use int 0x80.

read / write are both system calls.

read (. . .)

write (. . .)

Is there a way to make it simpler for assembly programmers?

Low-level programming

Java → Java Native Interface

→ A Java function is allowed to call a C function using this technique

CPython → You can call a C function

User program

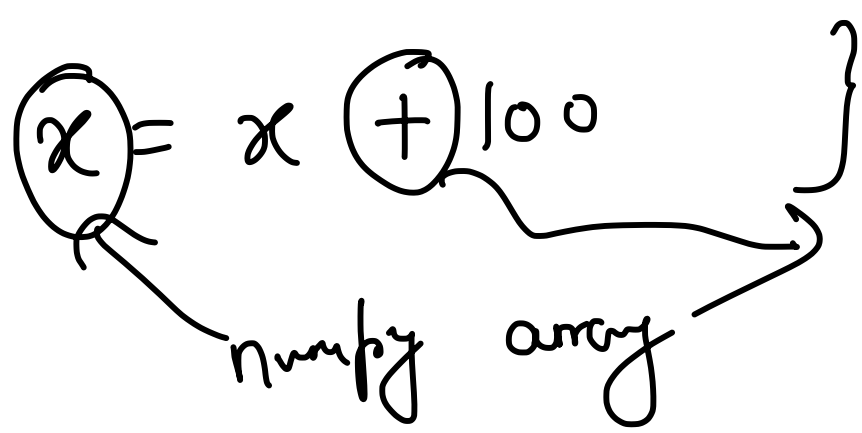
↓

Python function → interface } wrapper function

↓

C / C++ function

```
for (int i=0; i<100; i++) {
    x[i] = x[i] + 100;
}
```



calling a wrapper add function written in python, but itself calls another C++ function.

Since there is a write system call, there is also a write wrapper function.

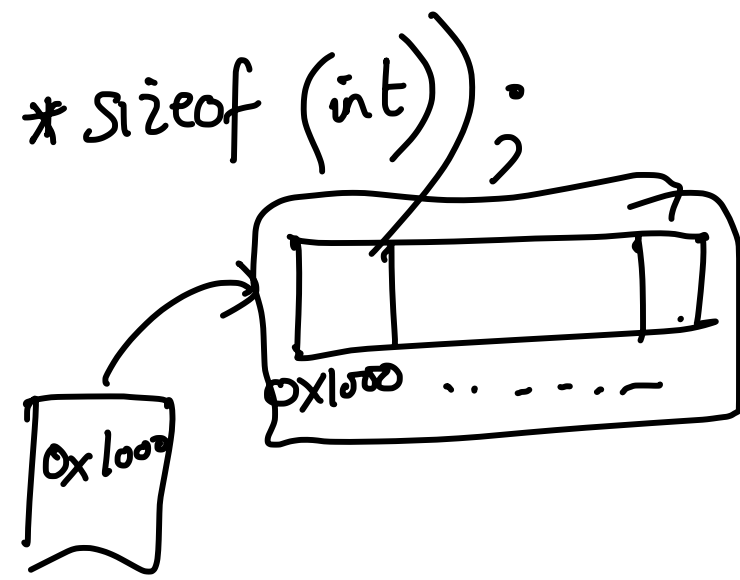
There is no need to remember for the

programmer that write is a system call; just using the wrapper function and using call is sufficient.

With ld command, need to link with the basic C libraries called glibc.

You can send output to the console using both printf and write functions using the same arguments.

```
void abc ( )      {
        int *a, b;
        a = (int *) malloc (100 * sizeof (int));
        free (a);
        return
}
int main ()       {
        abc ();
        :  <-- Reach here, but without having
        :      any chance of accessing the
```



}

memory block.

Memory leak

Responsibility of avoiding memory leaks is on the programmer; not on the assembler or compiler.

Is there a way to detect memory leak?

valgrind, etc to detect.

run-Time

There is no foolproof way of guaranteeing that your program has no memory leak, if no automatic garbage collection is present.